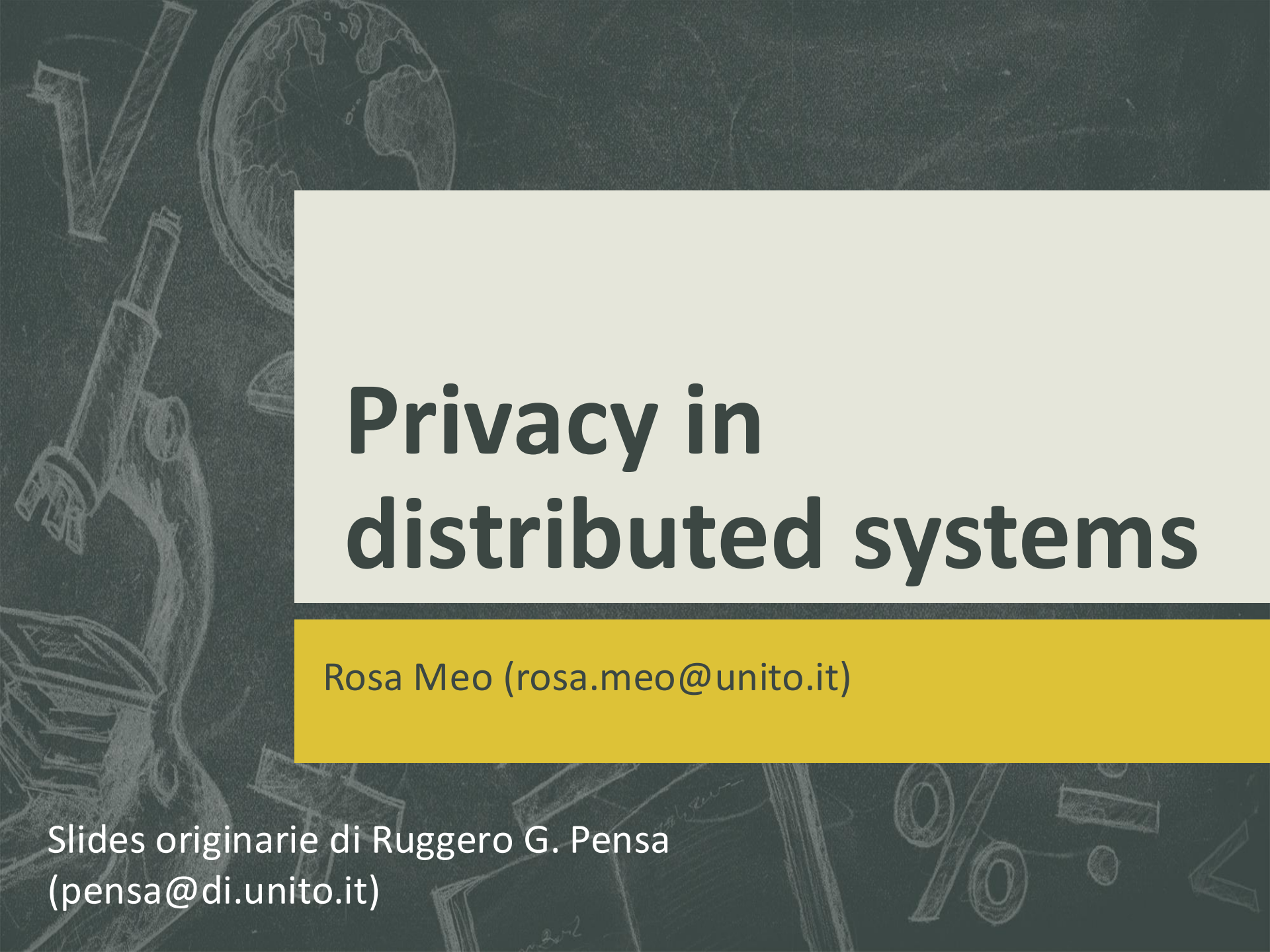




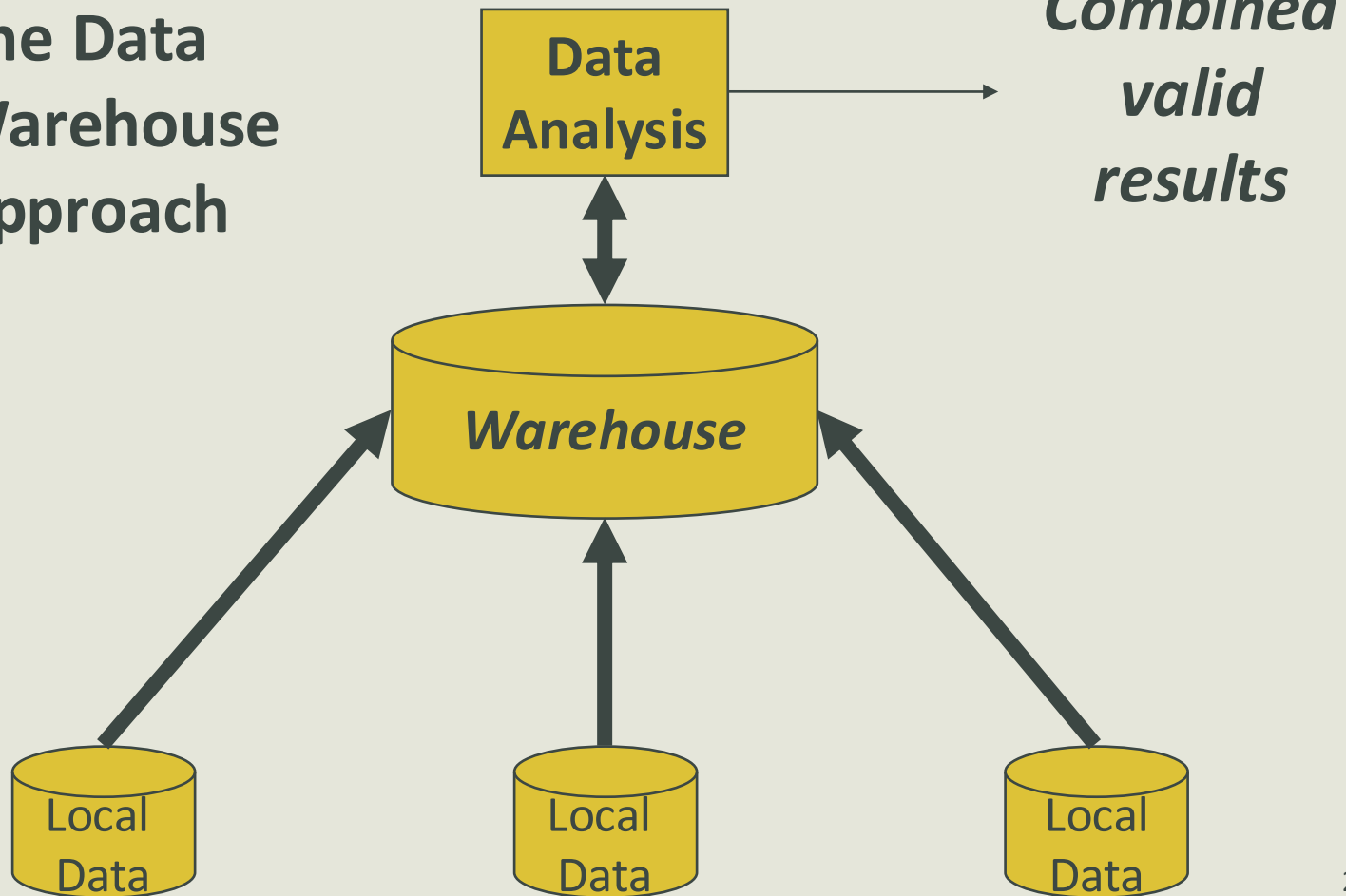
Privacy in distributed systems

Rosa Meo (rosa.meo@unito.it)

Slides originarie di Ruggero G. Pensa
(pensa@di.unito.it)

Distributed Data Analysis: The “Standard” Method

**The Data
Warehouse
Approach**



Distributed Data Analysis: The “Standard” Method

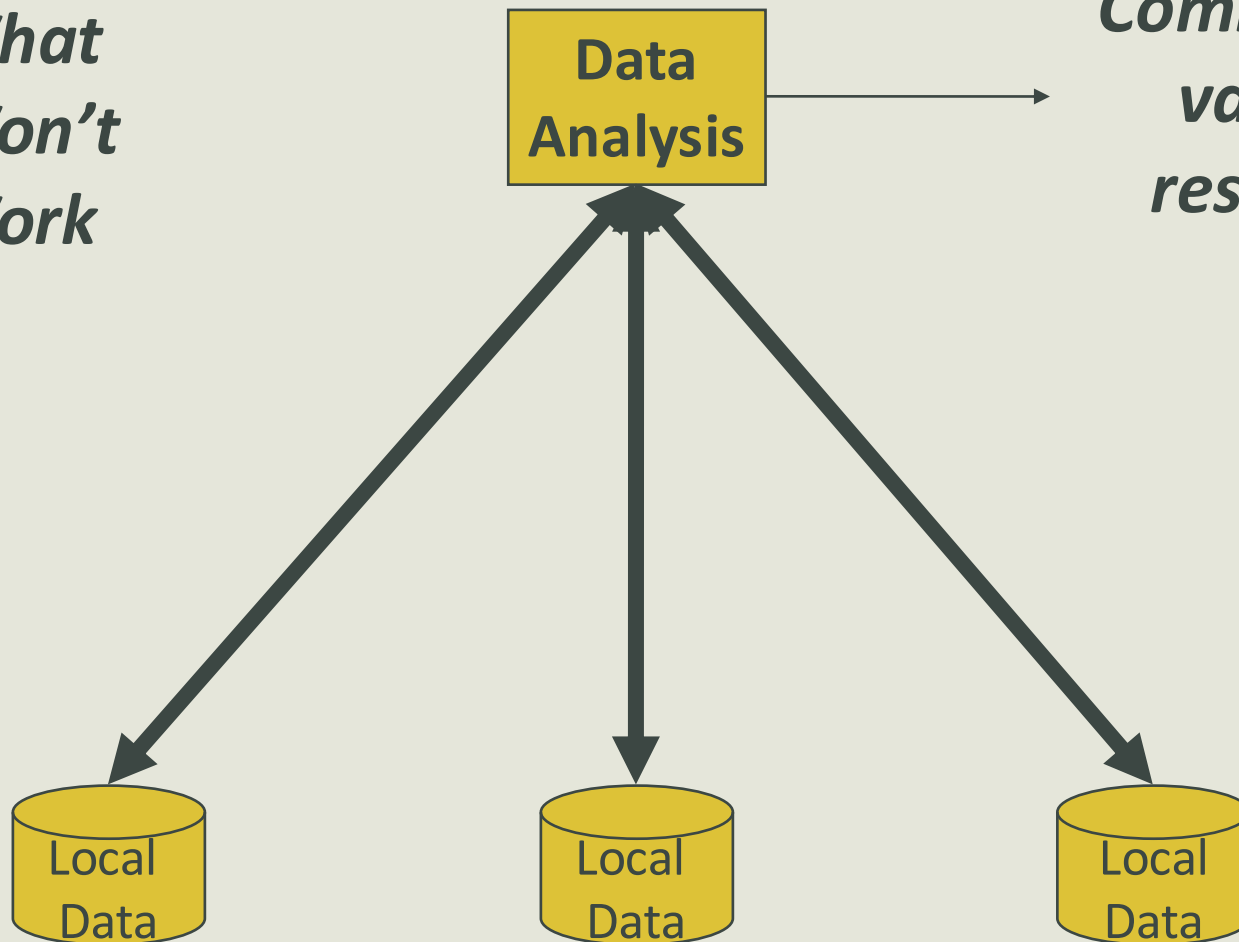
The traditional approach to analyzing data from distributed sources has been to gather all data into a central data warehouse, then run data mining/machine learning algorithms against the warehouse

There are a number of applications that are infeasible under such a constraint, leading to a need for distributed data analysis

- performance
- connectivity
- heterogeneity of sources
- privacy of sources

Private Distributed Data Analysis: What is it?

*What
Won't
Work*



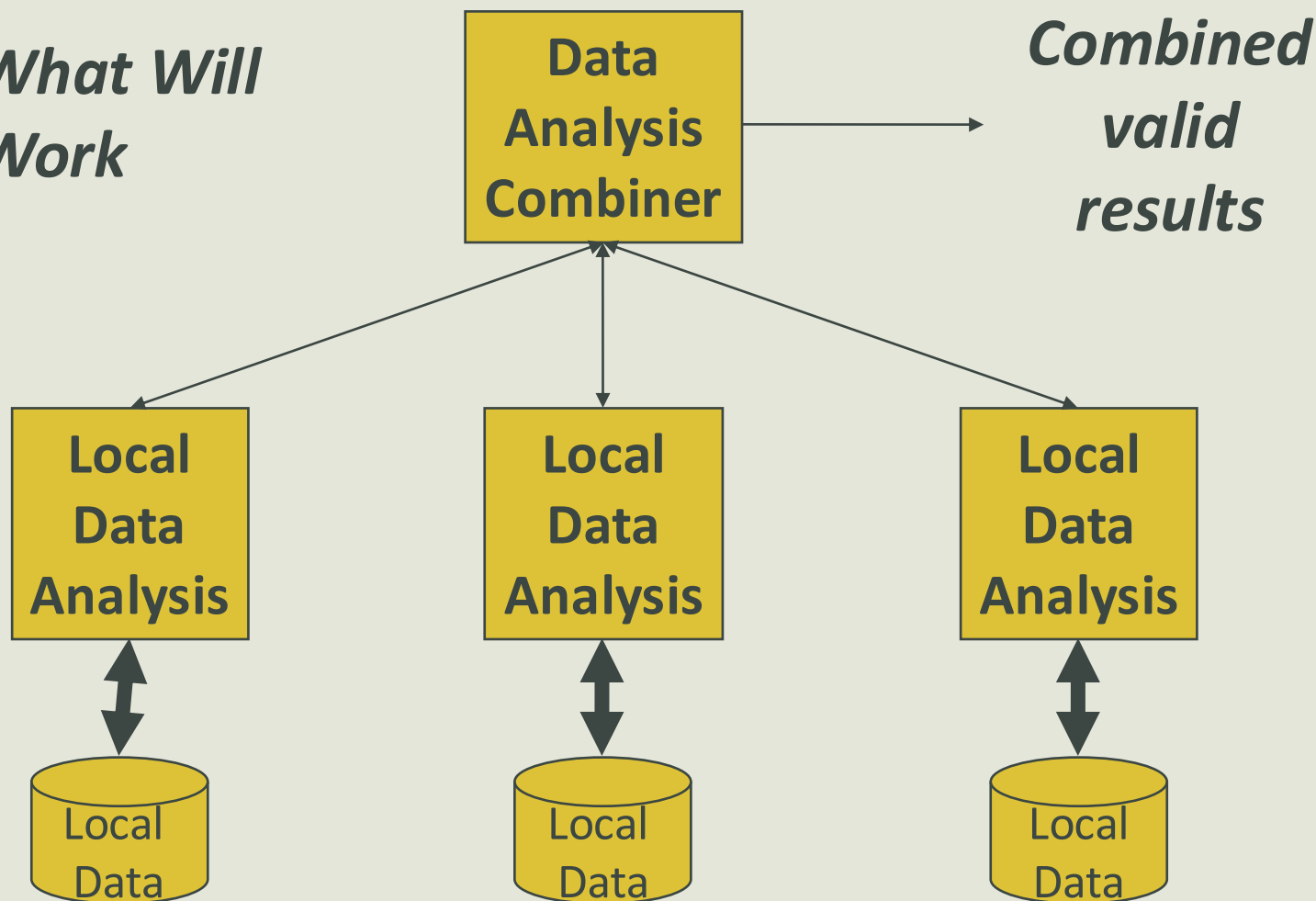
*Combined
valid
results*

Private Distributed Data Analysis: What is it?

- Simply creating a data mining/machine learning algorithm that extracts data directly from the local sources doesn't solve these problems
- All the issues that prevented creating a data warehouse still exist, they are just merged in with the data mining tool

Private Distributed Mining: What is it?

*What Will
Work*



Privacy-Preserving Data Analysis: Who?

- Government / public agencies
 - The Centers for Disease Control want to identify disease outbreaks
 - Insurance companies have data on disease incidents, seriousness, patient background, etc.
 - But can/should they release this information?
- Industry Collaborations / Trade Groups
 - An industry trade group may want to identify best practices to help members
 - But some practices are trade secrets
 - How do we provide “commodity” results to all (Manufacturing using chemical supplies from supplier X have high failure rates), while still preserving secrets (manufacturing process Y gives low failure rates)?

Privacy-Preserving Data Analysis: Who?

- Multinational Corporations
 - A company would like to mine its data for globally valid results
 - But national laws may prevent transborder data sharing
- Public use of private data
 - Big data analytics enables research studies of large populations
 - But these populations are reluctant to release personal information

Classes of Solutions

- Data Obfuscation
 - Nobody sees the **real** data
- Summarization
 - Only the needed facts are exposed
- Data Separation
 - Data remains with trusted parties

Data Separation

- Goal: Only trusted parties see the data
- Approaches:
 - Data held by owner/creator
 - Limited release to trusted third party
 - Operations/analysis performed by trusted party
- Problems:
 - Will the trusted party be willing to do the analysis?
 - Do the analysis results disclose private information?

Data Separation

- Data holders trusted with content
 - But only their own
- Mustn't share
 - But this doesn't prevent global models
- Solutions:
 - Secure Multiparty Computation
 - (Secure and private) federated learning

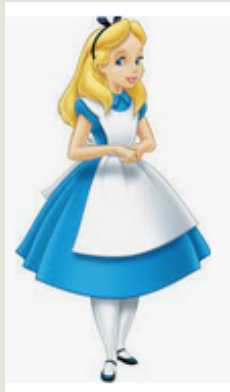
Secure Multiparty Computation

- Goal: Compute function when each party has some of the inputs
- Yao's Millionaire's problem
 - The problem discusses two millionaires, Alice and Bob, who are interested in knowing which of them is richer without revealing their actual wealth
 - The problem is solved by executing the function $(x > y)$ in a secure way with the function $(x > y)$ represented as a circuit
 - Idea: Securely compute gate
 - the circuit inputs are encrypted; the result of the circuit on encrypted inputs is still correct
 - The two parties use a key to encrypt their inputs without revealing their own input to the other party
- Works for multiple parties as well

Oblivious Transfer

- Oblivious Transfer (OT) is the fundamental cryptographic building block that allows Yao's Millionaires' Problem to be solved without a trusted third party
- It enables one millionaire (Alice) to create a secure circuit that both millionaires use to compute securely the same function on their inputs ($x > y$) without knowing the input of the other;
- The second millionaire (Bob) needs to obtain specific keys for the encryption of his input from the circuit creator, without revealing their own input or learning the other's total wealth.
- This is done with OT

Oblivious transfer (OT) 1 out of 2



Alice
(Sender)
has a pair of private inputs
 m_0 and m_1
(will be related to the
encryption keys)

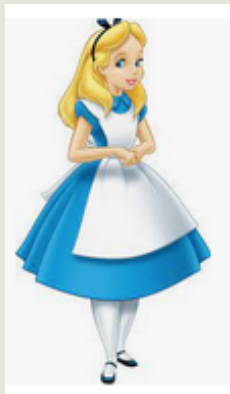


Bob
(Receiver)
has a private input $b=\{0,1\}$
that is a selection bit
(specifying which of the
two messages he would read)

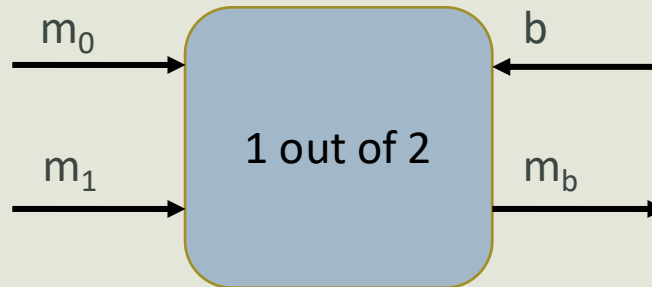
Security of the OT protocol

- Sender's security:
 - A corrupt receiver should learn no additional information as the message he wants to receive, or if he wants to receive message m_0 it should not be able to read also message m_1 (it will be unreadable with the received encryption key)
- Receiver's security:
 - A corrupt sender should learn no information on b
 - —→ this is why it is called oblivious (ignaro) protocol because the right message (encryption key) is sent, but not directly, and the sender does not know which of the two messages is read

Oblivious transfer (OT) 1 out of 2



Alice
(Sender)
has a pair of private inputs
 m_0 and m_1
(will be related to the
encryption keys)



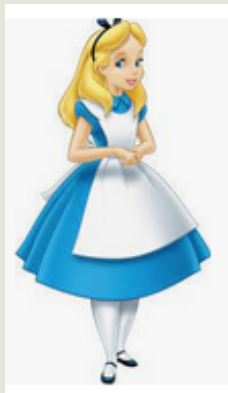
Bob
(Receiver)
has a private input $b=\{0,1\}$
that is a selection bit
(whose value corresponds to
one of the two messages
 m_0 and m_1)

Oblivious transfer (OT) 1 out of 2

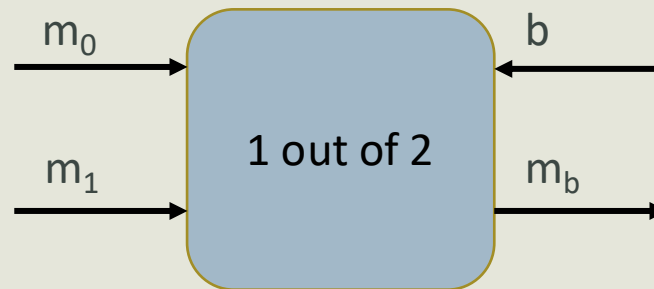
$$m_b = (1 \oplus b)m_0 \oplus bm_1$$

If $b=0 \rightarrow 1 \cdot m_0 \oplus 0 = m_0$

If $b=1 \rightarrow 0 \cdot m_0 \oplus 1 \cdot m_1 = 0 \oplus m_1 = m_1$



Alice
(Sender)
has a pair of private inputs
 m_0 and m_1

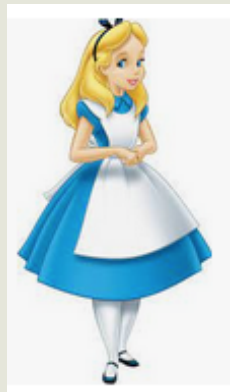


Bob
(Receiver)
has a private input $b=\{0,1\}$
that is a selection bit
(specifying which of the
two messages he would read)

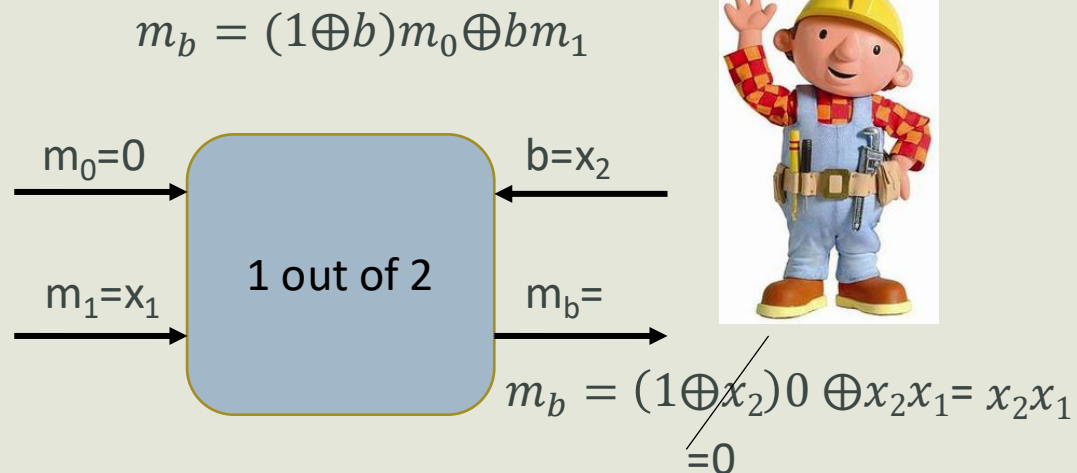


A	B	Q
0	0	0
0	1	1
1	0	1
1	1	0

A secure oblivious transfer (OT) 1 out of 2 can be used to compute a secure AND of their bits



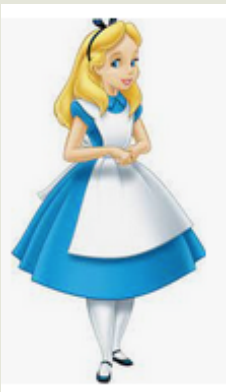
Alice
(Sender)
has a pair of private inputs



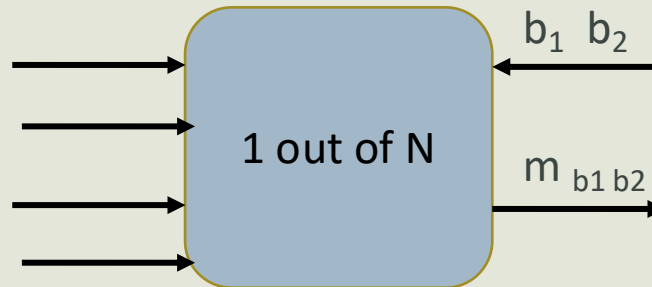
Bob
(Receiver)
has a private input b

Now we construct 1 out of N OT from 1 out of 2 OT

$N=4$



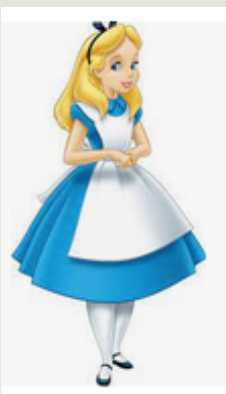
m_{00}
 m_{01}
 m_{10}
 m_{11}



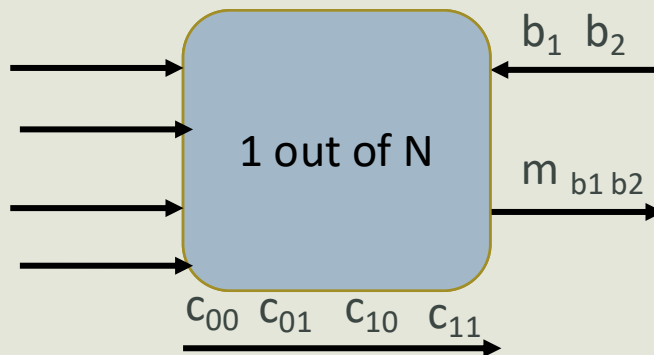
Alice
(Sender)
has $N=2^k$ private inputs
Must not know which message
Bob wants to read
She will send all of them,
but Bob will be able to read only one of them

Bob
(Receiver) wants to read one message only
has $k=2$ private inputs b_j
to indicate the indexes of the desired message

Now we construct 1 out of N OT from 1 out of 2 OT



m_{00}
 m_{01}
 m_{10}
 m_{11}

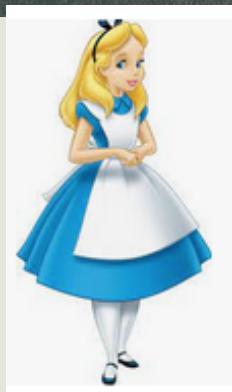


Alice
(Sender)
picks $k=2$ pairs of random keys
that will use to encrypt the messages

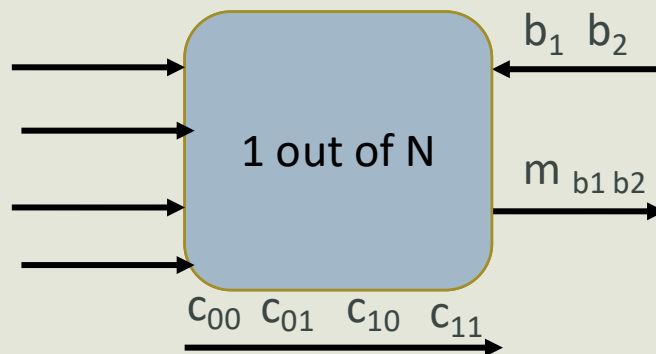
(k_0, k_1) for the LSB index
 (k'_0, k'_1) for the MSB index

Sender encrypt m_{ij} with pair of keys k'_i, k_j 21

Now we construct 1 out of N OT from 1 out of 2 OT



m_{00}
 m_{01}
 m_{10}
 m_{11}



Alice
(Sender)
picks $k=2$ pairs of random keys
that will use to encrypt the messages

(k_0, k_1) for the LSB index
 (k'_0, k'_1) for the MSB index

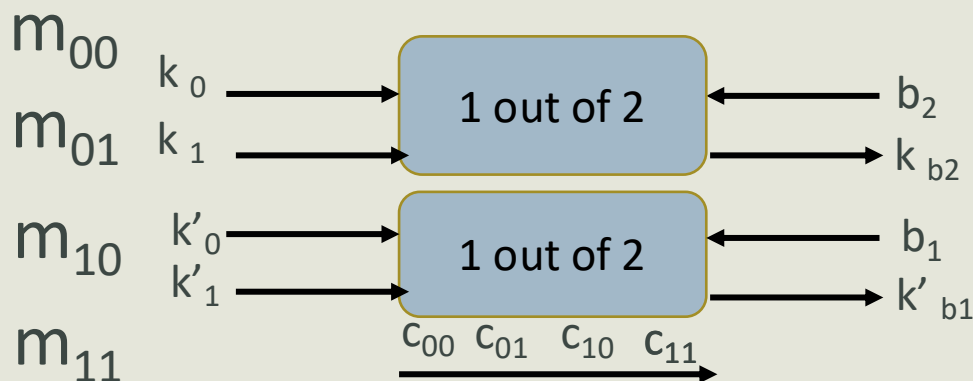
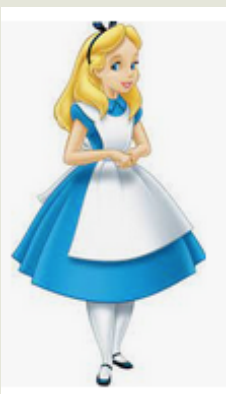
The receiver, depending on his choice bits
 $b_1 b_2$ will be able to decypher only one
of the N messages, only the one corresponding
to his choice bits.

Sender encrypt m_{ij} with pair of keys $k'_i k_j$ ²²
generating cyphertext c_{ij}

m_{10} $\xrightarrow{\text{Encrypt with } k'_1 k_0}$ c_{10}
 m_{11} $\xrightarrow{\text{Encrypt with } k'_1 k_1}$ c_{11}

m_{00} $\xrightarrow{\text{Encrypt with } k'_0 k_0}$ c_{00}
 m_{01} $\xrightarrow{\text{Encrypt with } k'_0 k_1}$ c_{01}

Now we construct 1 out of N OT from 1 out of 2 OT

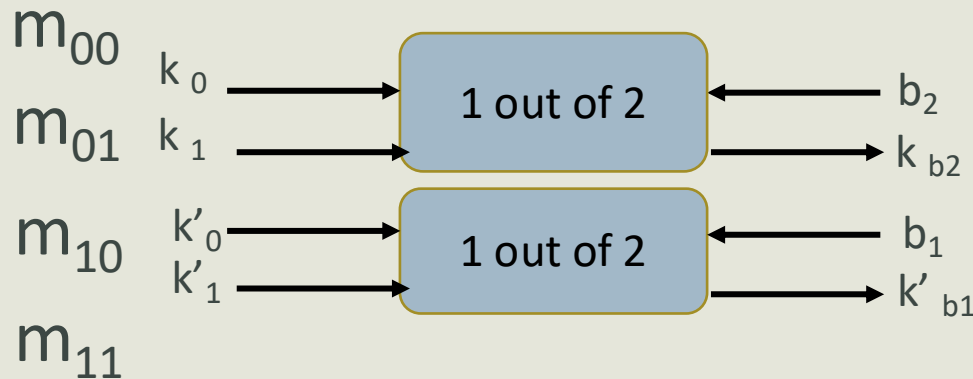
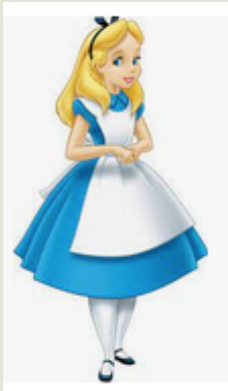


Alice
(Sender)
picks $k=2$ pairs of random keys
that will use to encrypt the messages

The receiver, depending on his choice bits b_1 b_2 will be able to decypher only one of the N messages, only the one he wishes to read. He receives the keys k_{b2} and k'_{b1} through 1 out of 2 OTs

(k_0, k_1) for the LSB index
 (k'_0, k'_1) for the MSB index $\longrightarrow$ Sender encrypt m_{ij} with pair of keys k'_i k_j 23

Now we construct 1 out of N OT from 1 out of 2 OT



$$c_{00} = F_{k'_0}(00) \oplus F_{k_0}(00) \oplus m_{00}$$

$$c_{01} = F_{k'_0}(01) \oplus F_{k_1}(01) \oplus m_{01}$$

$$c_{10} = F_{k'_1}(10) \oplus F_{k_0}(10) \oplus m_{10}$$

$$c_{11} = F_{k'_1}(11) \oplus F_{k_1}(11) \oplus m_{11}$$

with $F_k(\text{indexes})$ are pseudo-random functions (for symmetric encryption) with parameters (indexes) and a key, that imitates a random function when the key is not known

(k_0, k_1) for the LSB index

(k'_0, k'_1) for the MSB index

Sender encrypt m_{ij} with pair of keys k'_i, k_j 24

Oblivious polynomial evaluation

- The sender's input is a polynomial Q of degree k over some finite field F and the receiver's input is an element $z \in F$ (the degree k of Q is public).
- The protocol is such that the receiver obtains $Q(z)$ without learning anything else about the polynomial Q , and the sender learns nothing.
- The overhead of that protocol is $O(k)$ exponentiations (because it runs an exchange with the private-public key cryptographic techniques)
- This protocol maintains privacy in the face of a malicious adversary
- Our scenario in $ID3_\delta$ only requires a simpler protocol with semi-honest adversaries (honest but curious).
Such a protocol can be designed based on any homomorphic encryption scheme, with an overhead of $O(k)$ computation and $O(k|F|)$ communication.

Yao's two-party protocol (1)

- It is a constant-round (number of iterations) protocol for privately computing any probabilistic polynomial-time functionality (where the adversary may be semi-honest (honest-but-curious))
- Party 1 has input x and Party 2 input y
- f is the functionality they wish to compute
We assume that both parties wish to receive the same value $f(x,y)$
- $f(x,y)$ is evaluated privately and securely on both the inputs without knowing the other party's input

Yao's two-party protocol: how it works

- One of the parties (say Party 1) first generates an “encrypted” or “garbled” circuit computing $f(x, \cdot)$ and it sends it to Party 2
- The circuit does reveal nothing in its encrypted form and Party 2 learns nothing from this stage
- Party 2 obtains the output $f(x, y)$ by “decrypting” the circuit. This is accomplished by Party 2 obtaining from Party 1 a series of keys corresponding to its input y :
- This can be done by running $|y|$ instances of a private 1-out-of-2 Oblivious Transfer protocol
- Given these keys and the circuit, the output value $f(x, y)$ is obtained (and only it)

Overhead of the Yao's protocol

- (1) Party 1 (Alice) sends Party 2 (Bob) tables of size linear in the size of the circuit (each circuit node is assigned a table of keys for the decryption process - see table in later slide)
- (2) Party 1 and Party 2 engage in an oblivious transfer protocol for every input wire of the circuit ($|y|$ times)
- (3) Party 2 computes a pseudo-random function, a constant number of times for every gate (this is the cost incurred in decrypting the circuit).
- Therefore, the number of rounds of the protocol is constant (namely, two rounds using the oblivious transfer OT_1^2),
If the circuit is small (e.g., linear in the size of the input) then the main computational overhead is that of running the oblivious transfers.

How to solve the millionaire problem?

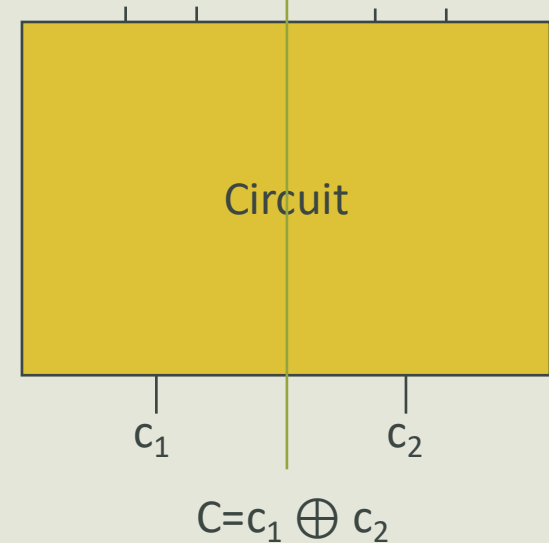
- To solve the Millionaire problem the wealth values are converted into binary strings.
- The parties (Alice and Bob) evaluate a comparison circuit (like a "greater-than" circuit) gate-by-gate for each bit of their binary string: $(A \oplus B) \cdot A$
- Each AND gate in that circuit is evaluated using the OT-table shown below to maintain the privacy of the original binary values (A and B).
- Each bit (A and B) is split into two "shares": A is split into a_1 and a_2 , such that $A = a_1 \oplus a_2$ and $B = b_1 \oplus b_2$ in which Alice adds a random bit c_1 .
- This is used as a cryptographic technique:
the secret binary values (A or B which are 0 or 1) are split into multiple random pieces (shares, a_i and b_i) such that the pieces can be combined to reconstruct the secret by the operation $\oplus$, the "addition" operation (modulo 2).

How does it work?

- Each side has input, knows circuit to compute function
 - Add random value to your input, give to other side
 - Each side has *share* of all inputs
- Compute share of output
 - Add results at end
- XOR gate: just add locally
- AND gate: send your share encoded in truth table
 - Oblivious transfer allows other side to get only correct value out of truth table

$$A = a_1 \oplus a_2$$

$$B = b_1 \oplus b_2$$



value of Bob (a_2, b_2)	(0,0)	(0,1)	(1,0)	(1,1)
OT-input (Alice)	1	2	3	4
value of output c_2 (Bob)	$c_1 \oplus a_1 b_1$	$c_1 \oplus a_1 (b_1 \oplus 1)$	$c_1 + (a_1 \oplus 1) b_1$	$c_1 + (a_1 \oplus 1) (b_1 \oplus 1)$

Why it works

- The goal is for Bob to receive a share c_2 such that when it is added to Alice's share c_1 , the result is the product of their combined inputs:

$$c_1 \oplus c_2 = (a_1 \oplus a_2) \cdot (b_1 \oplus b_2)$$

- By choosing the correct OT-input, Bob ensures that the mask c_1 is cancelled out, leaving the parties with valid shares of the AND result.
- For example, if Bob's input is $(a_2, b_2) = (1, 1)$, it selects the 4th value:

$$c_2 = c_1 \oplus (a_1 + 1) \cdot (b_1 + 1)$$

- When they later combine their shares:

$$c_1 \oplus c_2 = c_1 \oplus (c_1 \oplus (a_1 \oplus 1) \cdot (b_1 \oplus 1)) = (a_1 \oplus 1) \cdot (b_1 \oplus 1)$$

- This is exactly $(a_1 \oplus a_2) \cdot (b_1 \oplus b_2)$ because $(a_2, b_2) = (1, 1)$

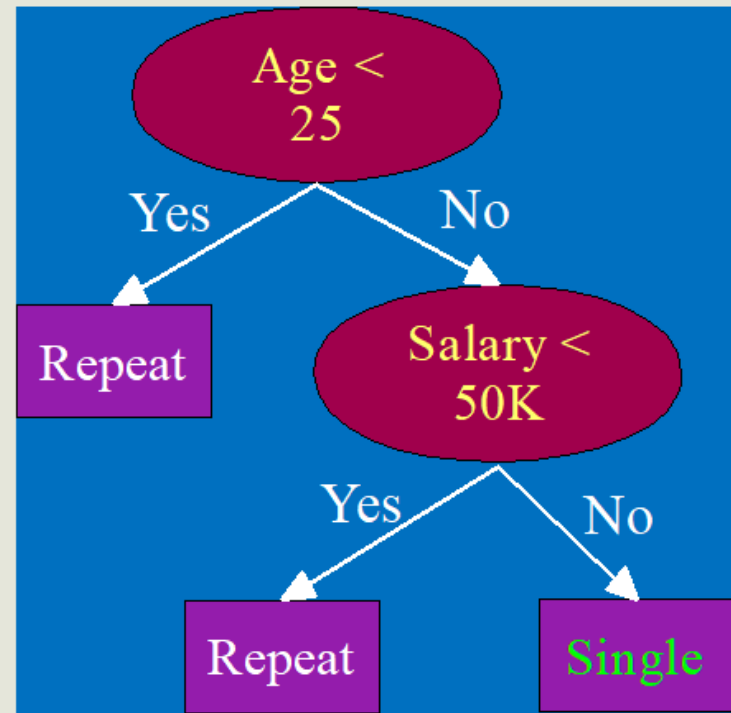
Decision Tree Construction

Y. Lindell, B. Pinkas:
Privacy Preserving Data Mining. J.
Cryptology 15(3): 177-206 (2002)

- Two-party horizontal partitioning
 - Each site has same schema
 - Attribute set known
 - Individual entities private
- Learn a decision tree classifier
 - ID3
- Essentially ID3 meeting Secure Multiparty Computation Definitions

Decision Tree Example

Age	Salary	Repeat Visitor?
23	50K	Repeat
17	30K	Repeat
43	40K	Repeat
68	50K	Single
32	70K	Single
20	20K	Repeat



ID3 algorithm (sketch)

1. Calculate the Entropy/Information Gain (IG) of each feature
2. If all rows don't belong to the same class, split the dataset into subsets using the feature for which the Entropy is minimum (or the IG is maximum)
3. Make a decision tree node using the feature with the minimum Entropy/maximum IG
4. If all rows belong to the same class, make the current node as a leaf node with the class as its label
5. Repeat for the remaining features until there are no features left, or the decision tree has all leaf nodes

Key Assumptions/Limitations

- Protocol takes place in the semi-honest model
 - Follows the protocol properly, but keeps a record of all its intermediate computations
- Only Two-party case considered
 - Extension to multiple parties is not trivial
- Computes an ID3 approximation ($ID3_{\delta}$)
 - any implementation of ID3 that computes the logarithm function to a predefined precision level has an approximation error that is the parameter δ
 - $ID3_{\delta}$ instead uses an approximation of Information Gain that does not use the computation of the log function and improves the efficiency
 - $ID3_{\delta}$ computes the set of decision trees having at nodes attributes that differ for information gain of less than δ (a low value)
- Deals only with categorical attributes

ID3 key steps

- A – the set of attributes
 - C – the class attribute
 - D – the set of data instances
1. Calculate the entropy/Information Gain of every attribute using the data set D
 2. Split the set D into subsets using the attribute for which the resulting entropy (after splitting) is minimum (or, equivalently, IG is maximum)
 3. Make a decision tree node containing that attribute
 4. Recurse on subsets using remaining attributes

Privacy Preserving ID3

Step 1: If A is empty, return a leaf-node with the class value assigned to the most transactions in D

- Set of attributes is public
 - Both know if A is empty
- Run Yao's protocol for the following functionality:
 - Inputs $(|D_1(c_1)|, \dots, |D_1(c_L)|), (|D_2(c_1)|, \dots, |D_2(c_L)|)$
(set of the number of examples in a partition of Party1 with the various classes, set of the same of Party 2)
 - Output i where $|D_1(c_i)| + |D_2(c_i)|$ is largest (most frequent class)
 - Computation is proportional to the dimension of the circuit and so in the number of classes and logarithm of the number of examples

Privacy Preserving ID3

Step 2: If D consists of examples which have all the same value c for the class attribute, return a leaf node with the value c

- In order to compute this step privately, represent leaf node having more than one class (in the example set), by a fixed symbol different from c_i ,
- Force the parties to input either this fixed symbol or c_i
- Check equality to decide if at leaf node for class c_i
- Various approaches for equality checking (could run Yao's algorithm on a circuit for equality checking)

Privacy Preserving ID3

- Step 3:
 - (a) Determine the attribute that best classifies the examples in D , let it be A_i
 - Essentially done by securely computing $x^*(\ln x)$
 - (b,c) Recursively call $ID3_\delta$ for the remaining attributes on the example sets $D(a_1), \dots, D(a_m)$ where $a_1, \dots, a_m$ are the values of the attribute A_i
 - Since the results of 3(a) and the attribute values are public, both parties can individually partition the database and prepare their inputs for the recursive calls

Private Protocol for approximating $X \ln X$

- Yao's protocol is used to obtain a very rough approximation to $\ln x$. Outputs from this step are random shares of the values n and ϵ such that

$$x = 2^n(1 + \epsilon) \text{ and } -1/2 < \epsilon < 1/2$$

Thus $n \ln 2$ is a rough estimate on $\ln x$ and $\ln(1 + \epsilon)$ is the “remainder”

- Value ϵ output from earlier step is used to privately compute Taylor series for $\ln(1 + \epsilon)$ to refine the computation. This involves polynomial evaluation of an integer polynomial that can be computed by simple circuits

Other (complementary) approaches

- Functional encryption
 - a generalization of public-key encryption in which possessing a secret key allows one to learn a function of what the ciphertext is encrypting
 - e.g., it enables data owners to grant third-party access to perform specified computations without disclosing their inputs
- (Fully) homomorphic encryption
 - a form of encryption that allows one to perform computations on encrypted data without first decrypting them
 - the resulting computations are left in an encrypted form
- Federated learning
 - allows one to train an algorithm across multiple decentralized edge devices or servers holding local data samples, without exchanging them